

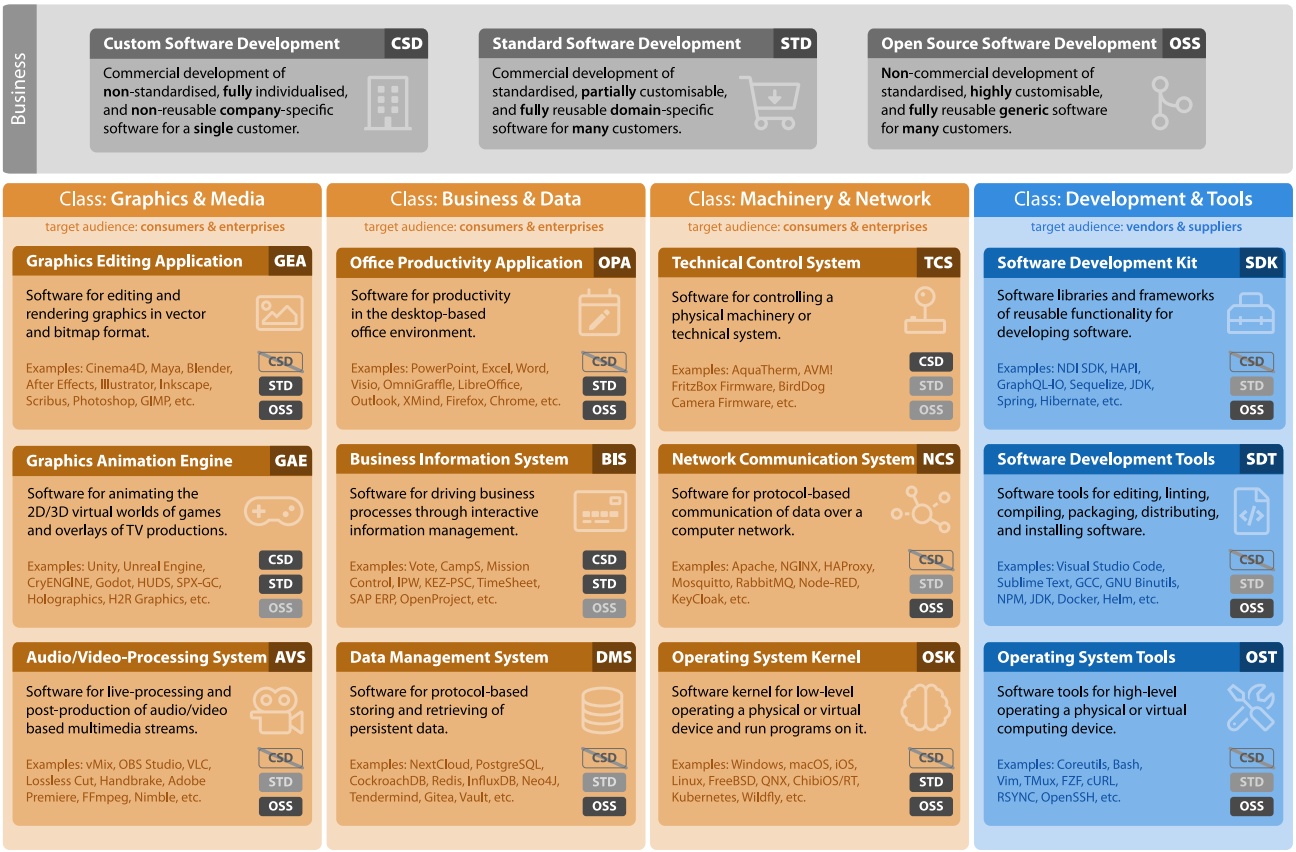
Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

ENGINEERING FUNDAMENTALS

Software Classes

TECHNISCHE UNIVERSITÄT MÜNCHEN



Intellectual Content: Version 1.1 (2022-12-10), Authored 2021-2022 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.1.0 (2022-12-10), Copyright © 2021-2022 Dr. Ralf S. Engelschall <http://engelschall.com>, All rights reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

EF 01.1

Es gibt drei traditionelle Ansätze der Software-Entwicklung: **Custom Software Development**, die kommerzielle Entwicklung von nicht-standardisierter, vollständig individualisierter und nicht wiederverwendbarer firmenspezifischer Software für einen einzelnen Kunden; **Standard Software Development**, die kommerzielle Entwicklung einer standardisierten, teilweise individualisierbaren, aber vollständig wiederverwendbaren fachlichen Software für viele Kunden; und **Open-Source-Software Development**, die nicht-kommerzielle Entwicklung einer standardisierten, hochgradig anpassbaren und vollständig wiederverwendbaren technischen Software für viele Kunden.

Es gibt vier Bereiche und 12 Klassen von Software. Im ersten Bereich gibt es drei Klassen von Software, die sich auf Grafik & Medien fokussieren: **Graphics Editing Application**, Software zum Bearbeiten und Rendern von Grafiken im Vektor- und Bitmap-Format; **Graphics Animation Engine**, Software für die Animation der virtuellen 2D/3D-Welten von Spielen und der Overlays von TV-Produktionen; und **Audio/Video-Processing Systems**, Software für die Live-Bearbeitung und Nachbearbeitung von Audio/Video-Multimedia-Streams.

Im zweiten Bereich gibt es drei Klassen von Software, die sich auf Geschäft & Daten fokussieren: **Office Productivity Application**, eine Software für Produktivität in der Desktop-basierten Büroumgebung; **Business Information System**, eine Software zur Steuerung von Geschäftsprozessen mittels Informationsmanagement; und **Data Management System**, eine Software zum Speichern und Abrufen von persistenten Daten.

Im dritten Bereich gibt es drei Klassen von Software, die sich auf Maschinen & Netzwerk fokussieren: **Technical Control System**, eine Software zur Steuerung einer physischen Maschine oder eines technischen Systems; **Network Communication System**, eine Software für die Kommunikation von Daten über ein Computer-Netzwerk; und **Operating System Kernel**, ein Software-Kern für den Betrieb eines physischen oder virtuellen Computers auf einer niedrigeren Ebene.

Im vierten Bereich gibt es drei Klassen von Software, die sich auf Entwicklung & Werkzeuge fokussieren: **Software Development Kit**, Software-Bibliotheken und -Frameworks mit wiederverwendbarer Funktionalität für die Entwicklung von Software; **Software Development Tools**, Software-Werkzeuge zum Bearbeiten, Analysieren, Übersetzen, Paketieren und Installieren von Software; und **Operating System Tools**, Software-Werkzeuge für den Betrieb eines physischen oder virtuellen Computers.

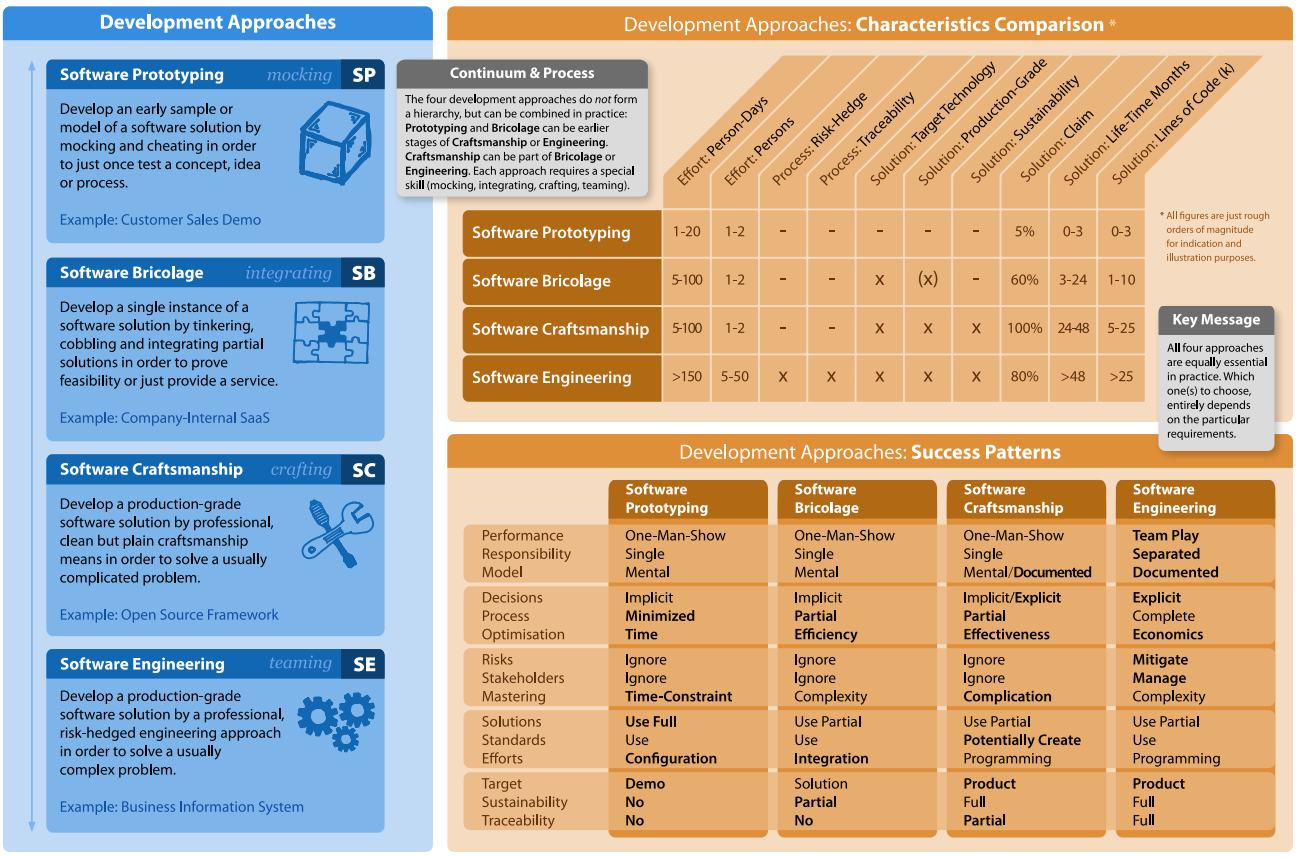
Fragen

Welche beiden Klassen von Software werden vor allem mit Hilfe des Ansatzes **Custom Software Development** entwickelt?

ENGINEERING FUNDAMENTALS

Software Development Approaches

TECHNISCHE UNIVERSITÄT MÜNCHEN



Intellectual content: Version 1.0.4 (2021-11-10), Authored 2021/2022 by Dr. Ralf S. Engelschall (Graphical illustration: Version 1.0.3 (2021-11-25)) Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com> – All Rights Reserved. Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contents only.

Man kann vier Arten von Software-Entwicklungs-Ansätzen unterscheiden.

Bei **Software Prototyping** entwickelt man ein frühes Beispiel oder Modell einer Softwarelösung durch “Mocking” und “Cheating”, um ein Konzept, eine Idee oder einen Prozess einmalig zu testen.

Bei **Software Bricolage** entwickelt man eine einzelne Instanz einer Softwarelösung durch Basteln, Zusammenschustern und Integrieren von Teillösungen, um eine Machbarkeit zu beweisen oder einfach nur eine Dienstleistung zu erbringen.

Bei **Software Craftsmanship** entwickelt man eine produktionsreife Softwarelösung mit professionellen, sauberen, aber rein handwerklichen Mitteln, um ein meist kompliziertes Problem zu lösen.

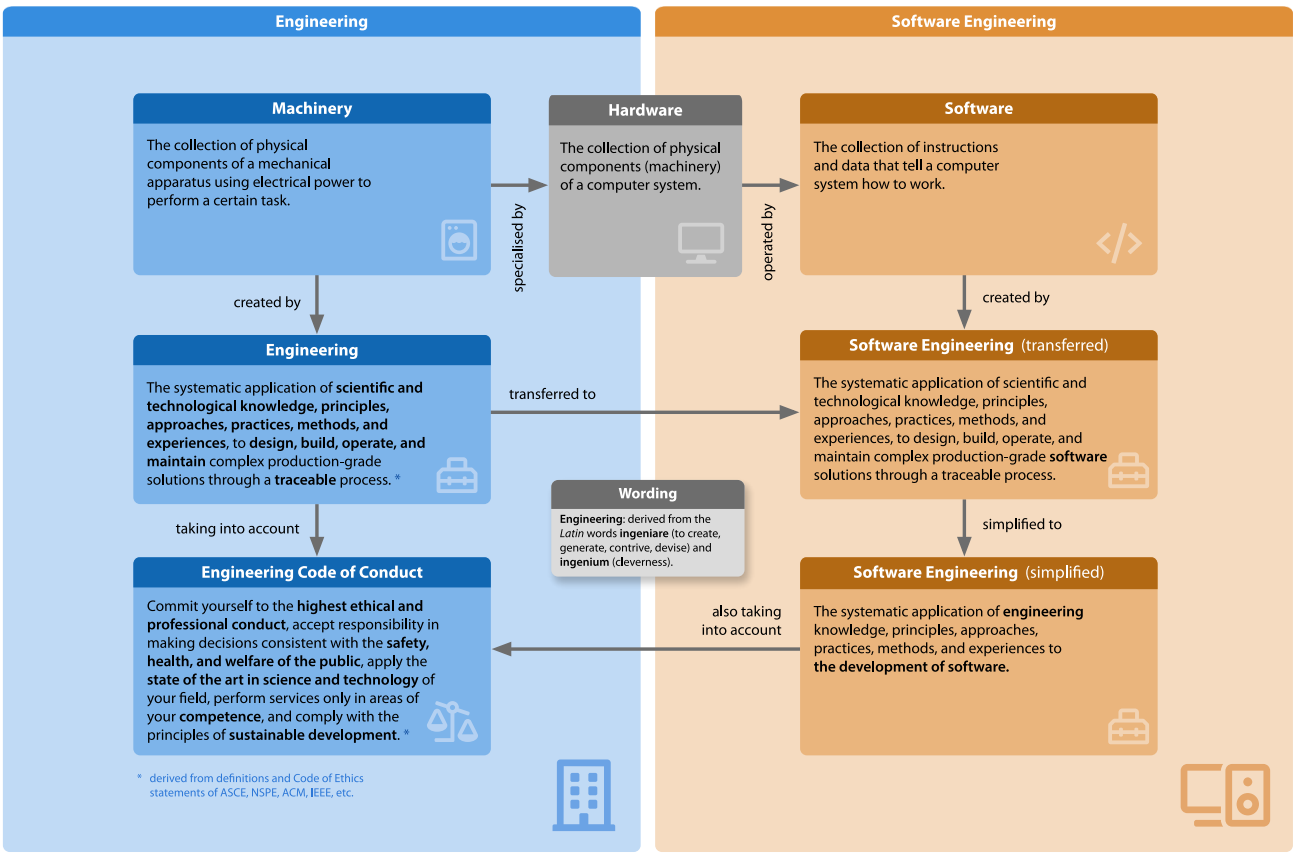
Bei **Software Engineering** entwickelt man eine produktionsreife Softwarelösung mit einem professionellen, risikoabsichernden Ingenieurs-Ansatz, um ein in der Regel komplexes Problem zu lösen.

Die vier Entwicklungsansätze können in der Praxis auch kombiniert werden: Prototyping und Bricolage können Vorstufen von Craftsmanship oder Engineering sein. Craftsmanship kann Teil des Engineering sein. Jeder Ansatz erfordert eine spezielle Fähigkeit. Alle vier Ansätze sind in der Praxis gleichermaßen wichtig. Welche(n) man jeweils wählt, hängt ganz von den konkreten Anforderungen ab.

Fragen

- Welchen Software-Entwicklungs-Ansatz sollte man wählen, um ein komplexes betriebliches Informationssystem zu realisieren?
- Welchen Software-Entwicklungs-Ansatz sollte man wählen, um eine komplizierte wiederverwendbare Bibliothek zu realisieren?

Software Engineering



Intellectual Content: Version 1.0.2 (2021-04-26), Authored 2021 by Dr. Ralf D. Engelken
Graphical Illustration: Version 1.0.2 (2021-04-26), Copyright © 2021 Dr. Ralf D. Engelken (http://engelken.com). All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contents only.

Ingenieurwesen (Engineering) ist die systematische Anwendung von wissenschaftlichem und technologischem Wissen, Prinzipien, Ansätzen, Praktiken, Methoden und Erfahrungen, um komplexe, produktionsreife Lösungen durch einen nachvollziehbaren Prozess zu entwerfen, zu bauen, zu betreiben und zu warten.

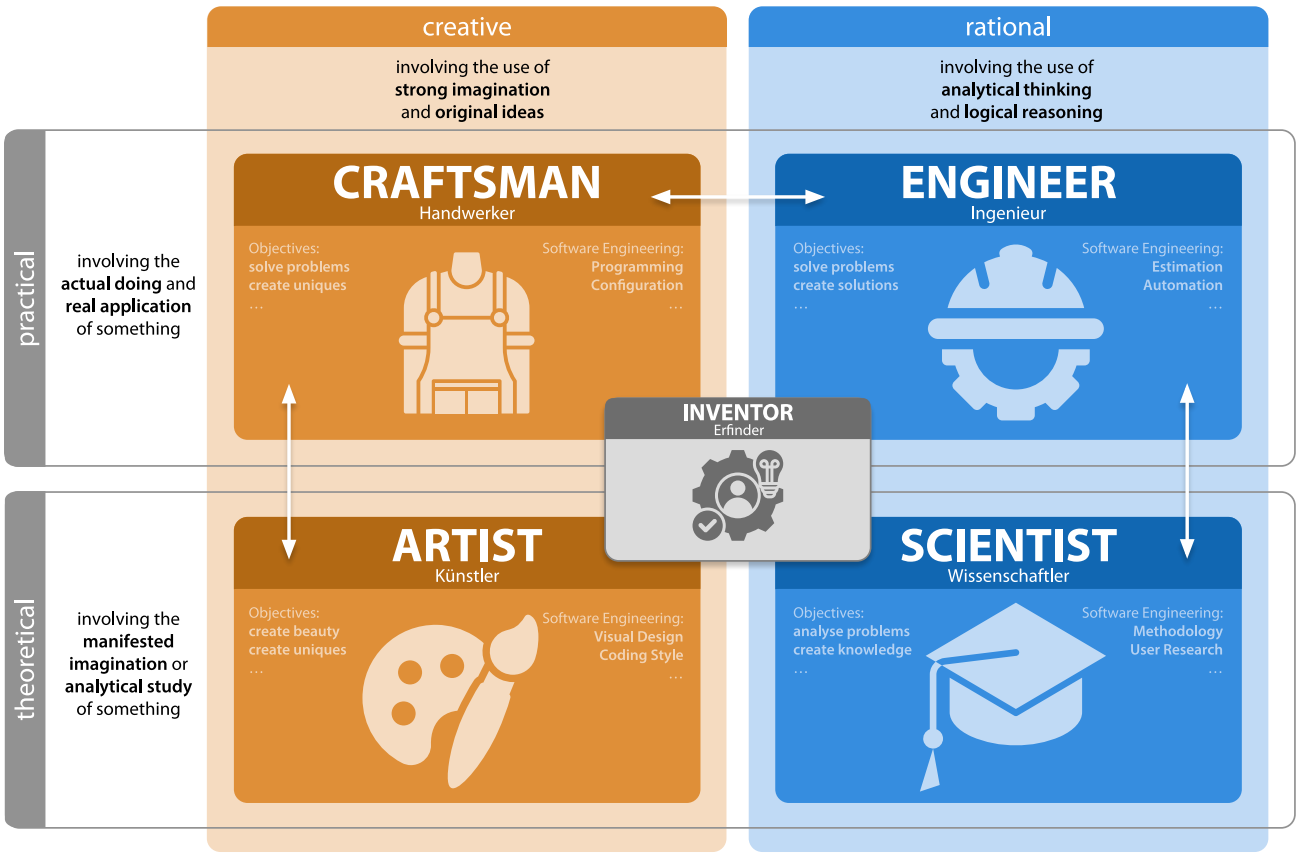
Software Engineering ist die systematische Anwendung von ingenieurwissenschaftlichem Wissen, Prinzipien, Ansätzen, Praktiken, Methoden und Erfahrungen auf die Entwicklung von Software.

Sowohl für das Engineering als auch für das Software Engineering gilt der folgende **Verhaltenskodex** (Code of Conduct): Verpflichten Sie sich zu höchstem ethischen und professionellen Verhalten; übernehmen Sie Verantwortung, Entscheidungen im Einklang mit der Sicherheit, der Gesundheit und dem Wohlergehen der Öffentlichkeit zu fällen; wenden Sie den Stand der Wissenschaft und Technik Ihres Fachgebiets an; erbringen Sie Leistungen nur in Bereichen Ihrer eigenen Kompetenz; und beachten Sie die Prinzipien der nachhaltigen Entwicklung.

Fragen

- Ist Software Engineering auch für die Entwicklung einer nicht-komplexen Software in einem kleinen Team von zwei Personen sinnvoll?

Profession Characteristics



EF 01.4
Intellectual Content: Version 1.0.0 (2024-04-10) Authored 2024 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.0 (2024-04-10) Copyright © 2024 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science Lecture contexts only
diagram.direct / XIRF-NIIV

Berufe haben in der Regel zwei von vier Merkmalen: **Kreativ** zu sein bedeutet, daß man eine starke Vorstellungskraft und originelle Ideen nutzt. **Rational** zu sein bedeutet, daß man analytisches Denken und logisches Schlussfolgern anwendet. **Praktisch** zu sein bedeutet, daß man etwas tatsächlich tut und anwendet. **Theoretisch** zu sein bedeutet, daß man eine Vorstellung einer Sache umsetzt oder eine Sache analytisch untersucht.

Man kann fünf interessante Berufe unterscheiden: Ein **Handwerker** handelt auf kreative und praktische Weise und löst Probleme und schafft Unikate. Ein **Ingenieur** handelt rational und praktisch und löst Probleme und schafft Lösungen. Ein **Künstler** handelt kreativ und theoretisch und erschafft Schönheit und Unikate. Ein **Wissenschaftler** handelt rational und theoretisch und analysiert Probleme und schafft Wissen. Ein **Erfinder** hingegen muss normalerweise alle Eigenschaften in sich vereinen.

Fragen

- Wenn man sich um Konfiguration und Programmierung im Software-Engineering kümmert, agiert man statt als Ingenieur eher wie ein…?

TRUE Manifesto

true-manifesto.org



Freiknecht, Jonas: Version 1.0.7 (2021-10-25). Authored 2015-2021 by Dr. Ralf S. Engelschall, Christian Reiber, Rudolf Kreppe and Matthias Bassalo.
Gesellschaft, Creation, Version 1.0.7 (2021-10-25). Copyright © 2015-2021 Dr. Ralf S. Engelschall <http://engelschall.com>. All rights reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Das TRUE Manifesto gibt die Grundhaltung für gutes Software Engineering als zentrales Manifest, ähnlich dem Ansatz – aber absichtlich in teilweisem Gegensatz zum Inhalt – des populären Agilen Manifests. Das TRUE Manifesto drückt vier Hauptaspekte aus.

(T)argeted (zielgerichtet) (angemessen, passend, fokussiert): Wir fokussieren uns auf angemessene und passende Lösungen und Vorgehen, denn: Sowohl Lösungen als auch Vorgehen müssen in einem vernünftigen Verhältnis zum Problem stehen.

(R)easoned (durchdacht) (wohlüberlegt, bewertet, willentlich): Wir denken im Vorfeld sorgfältig über unsere geplanten Lösungen und Vorgehen nach, denn: Denken ist billiger als Korrigieren.

(U)p-to-date (zeitgemäß) (ausgebildet, erfahren, beharrlich): Wir entwickeln hochqualitative Lösungen auf der Basis zeitgemäßer Methoden und Technologien, denn: Wir müssen mit der Tatsache zurechtkommen, daß die IT-Welt sich selbst wiederkehrend umwälzt.

(E)volutionary (evolutionär) (nachhaltig, harmonisch, kontextuell): Wir entwickeln nachhaltige Lösungen, die sich optimal in ihren Kontext einfügen, denn: Die Natur lehrt uns, daß nur evolutionäre Ansätze und Lösungen eine gute Chance haben, langfristig zu überleben.

Fragen

- Ist ein extrem agiles Vorgehen mit ständigem Refactoring im Sinne des TRUE Manifesto für Software Engineering?

ENGINEERING FUNDAMENTALS

Discipline Claim

TECHNISCHE UNIVERSITÄT MÜNCHEN

EF 01.6

FA FARSIGHTED weitblickend

Be farsighted in your solution finding.

Sei weitblickend in deiner Lösungsfindung.

AR: Scalable Hub'n'Spoke
DV: Plugin SPI



TE TENET-ORIENTED grundsatzorientiert

Orientate yourself on fixed tenets in your approach and solution finding.

Orientiere dich an festen Grundsätzen in deinem Vorgehen und deiner Lösungsfindung.

AR: Separation of Concern
DV: Strict Coding-Style



TH THOUGHTFUL wohlüberlegt

Act thoughtful in your approach and solution finding.

Agiere wohlüberlegt in deinem Vorgehen und deiner Lösungsfindung.

AR: Modularization
DV: Algorithmical Control Structure



HO HOLISTICALLY ganzheitlich

Think holistically and in the long-term when finding your solutions.

Denke ganzheitlich und langfristig in deiner Lösungsfindung.

AR: Walking Skeleton Design
DV: Consistent Error Handling



AD ADEQUATE angemessen

Ensure that your approach and solutions are adequate to the boundary conditions.

Sorge dafür, daß dein Vorgehen und deine Lösungen angemessen zu den Rahmenbedingungen sind.

AR: No Cloud-Native Complexity
DV: No Over-Engineered Abstractions



FE FEASIBLE machbar

Ensure that your approach and solutions can be realised at reasonable costs.

Sorge dafür, daß dein Vorgehen und deine Lösungen mit vernünftigen Kosten realisiert werden können.

AR: Existing Framework Functionality
DV: Realistic Programming Model

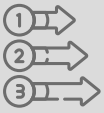


IN INCREMENTAL inkrementell

Apply the depth of your discipline incrementally.

Wende die Tiefe deiner Disziplin inkrementell an.

AR: Identified Solution Cruxes
DV: Minimum Viable Product



VA VALUEABLE wertvoll

Provide clearly recognizable added values with your approach and solutions.

Liefere klar ersichtliche Mehrwerte mit deinem Vorgehen und deinen Lösungen.

AR: Technology Stack Design
DV: User-Story-Driven Functionality



SU SUSTAINABLE nachhaltig

Create sustainable solutions that are well integrated into their environment.

Erschaffe nachhaltige Lösungen, die gut in ihre Umgebung integriert sind.

AR: Interoperable Interfaces
DV: Maintainable Code



Intellectual Content: Version 1.0.1 (2024-04-27), Authored 2022-2024 by Dr. rer. nat. S. Eder et al.
Graphical Illustration: Version 1.0.1 (2024-04-27), Copyright © 2022-2024 Dr. rer. nat. S. Eder et al., https://creativecommons.org/licenses/by-nc-nd/4.0/, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Über die verschiedenen **Disziplinen** des Software-Engineering hinweg gibt es einige gemeinsame Eigenschaften, die sie alle vom konzeptionellen Standpunkt aus beanspruchen: **weitblickend**, **grundsatzorientiert**, **wohlüberlegt**, **ganzheitlich**, **angemessen**, **machbar**, **inkrementell**, **wertvoll** und **nachhaltig**.

Fragen

- Was ist der schwierigste Anspruch in einer **Disziplin** in der heutigen Praxis?

ENGINEERING FUNDAMENTALS

Sustainability

TECHNISCHE UNIVERSITÄT MÜNCHEN



EF 01.7
Intellectual Content: Version 1.0.0 (2022-10-11) Authored 2022 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.0 (2022-10-11) Copyright © 2022 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only

Nachhaltiges Handeln sollte selbstverständlich sein, da es immer andere nach uns geben wird. Im Software Engineering bieten sich die folgenden Minimierungs-Prinzipien an, um nachhaltig zu agieren:

Minimierung des Leerlaufs und Maximierung der Nutzung der vorhandenen Hardware-Ressourcen; Minimierung der Ineffizienz und Maximierung der Effizienz von Software-Anwendungen und deren Entwicklungsprozessen; Minimierung der Gesamtmenge der gesammelten, übertragenen, gespeicherten und verbreiteten Informationen; Minimierung der Exzessivität und Maximierung der Angemessenheit von Lösungsdesigns; Minimierung der Vergänglichkeit und Maximierung der Lebensdauer jeglicher Art von Lösungen; Minimierung der Ausbeutung und Maximierung des Rückflusses in allen Arten von Ökosystemen; Minimierung des Einsatzes von Menschen und Maximierung des Einsatzes von Maschinen in allen Produktions- und Betriebsprozessen; Minimierung des Verbrauchs und Maximierung der Einsparung von Energie in allen Produktions- und Betriebsprozessen; und: Minimierung der Kohlenstoffemissionen und somit des Fußabdrucks bei allen Produktions- und Betriebsprozessen.

Fragen

- Ist die bewährte Praktik der Continuous Integration (CI) ein nachhaltiges Handeln?